

FoldBot ROS 2 行为树设计文档

把现有折叠规划逻辑抽出来，整理成可直接照着实现的 ROS 2 行为树基线：前方待洗篮取衣 → 中央折叠 → 后方四箱分拣堆放。

4 ROS 2 层 感知 / 规划 / 执行 / 编排	3 阶段路线 分拣 → 辅助折叠 → 全自主	100% 现有纯逻辑复用 planner / fold 接口不变	BT.CPP 4 行为树引擎 XML 可直接加载
--	---	---	---

一句话定位：本文档冻结接口与行为树结构。感知换数据源、执行换 MoveIt 2，折叠几何一行不改。后续真机开发只填实现，不再做架构决策。

版本	基线代码	目标平台
v1.0 · 2026-10-05	WanruJia/foldbot (main)	ROS 2 Humble / Jazzy + MoveIt 2
行为树	工作空间	双臂基准
BehaviorTree.CPP 4.x	前篮 → 中央台 → 后四箱	2x 6/7DoF + 平行夹爪 + RGB-D

00 目录与设计目标

01	现有资产盘点：什么直接搬，什么必须重写	3
02	总体架构：ROS 2 节点拓扑与数据流	4
03	行为树主树：任务循环、安全守卫与恢复	5
04	行为树节点目录：每个叶子节点的契约	6
05	接口契约：msg / srv / action 先行冻结	7
06	感知子系统：从颜色分割到 RGB-D 学习型关键点	8
07	折叠规划子系统：planner.js 的 1:1 移植	9
08	运动执行子系统：MoveIt 2 + 力控收尾	10
09	工作空间、坐标系与 TF 树	11
10	异常恢复与安全设计	12
11	三阶段路线：仿真到真机的收敛路径	13
12	里程碑与验收标准	14
A	附录 A：行为树 XML 骨架（可直接加载）	15
B	附录 B：ROS 2 包结构与文件迁移对照	16

设计目标（本期必须达成）

- **单件闭环**：从前方待洗篮取 1 件已铺平的衣服，在中央台完成折叠，按家人分类放入后方对应衣箱并码正堆叠。
- **逻辑零改动移植**：`planFolds / planPantsFolds / classifyOwner / detectPantsLength / detectKind` 的输入输出结构保持不变，仅换语言实现（JS → C++/Python）。
- **可单独测试**：每个行为树子树（取衣、感知、折叠、归位）可脱离整机单独跑通，支持假数据注入。
- **失败可恢复**：抓空、关键点缺失、规划失败三类高频问题各有明确的重试上限与降级路径，不无限循环。

非目标（明确不做）

- 任意乱衣堆的抓取与铺平（放到第三阶段）；本期衣服以大致铺平状态送入待洗篮。
- 多件并行折叠；双臂只在单件内分工（一臂抓取、一臂按压）。
- 移动底盘导航；机械臂固定安装，工作空间靠 TF 标定解决。

关键判断：真机最难的不是折叠规划，而是 deformable object 的抓取与滑移。本设计把规划当作已解决问题，把 80% 的工程量压在感知置信度、抓取重试与力控收尾上。

01 现有资产盘点：什么直接搬，什么必须重写

FoldBot 现有代码已经把“做什么”与“怎么动”分得很干净。纯数学模块（无 THREE 依赖、可 Node 单测）就是真机要复用的核心资产；Three.js 渲染与合成纹理部分在真机上没有对应物，直接退役。

现有文件	真机对应物	策略	说明
fold.js sideOfLine / foldPointVert	foldbot_planning 几何内核 (C++)	1:1 移植	Rodrigues 旋转与折叠侧判定是纯几何。折痕线定义、折叠方向与现有单测完全一致。
planner.js planFolds / planPantsFolds	planning_node 的 PlanFold 服务	1:1 移植	输出 folds[] / markers / up / right 直接映射为 ROS 2 消息。阈值改为参数文件。
kinematics.js ik / fk 双连杆解析解	Movelt 2 IK + 解析解作校验	降级复用	真机 6/7 轴臂不再用解析 IK 做主规划；保留做可达性预检，避免不可达目标发到 Movelt 才失败。
vision.js 凸包 / 凹点 / findKeypoints	perception 的 fallback 路径	保留 fallback	纯色 + 固定光照下仍可用；作为学习模型低置信度时的降级，并提供关键点格式黄金标准。
garments.js OWNERS / KINDS / 尺 寸先验	owners.yaml + 仿真资产	参数化	四位家人尺寸先验、裤长模式 (tri/bi/none) 从常量变为可标定参数。
app.js / Three.js	Gazebo / Isaac Sim + RViz	退役替换	“IK 算角度、衣服顶点跟着走”在真机不成立，这是要填的最大 gap，见第 08 节。

移植原则：凡是现有单测覆盖的纯函数，真机实现必须通过同一批用例的等价翻译版。行为树的任何重构都不许改变这批函数的输入输出。

折叠计划数据结构（真机原样保留）

```
plan = { folds: [ { params:{px,pz,dx,dz,foldSign,axisH,duration},  
  label, status, grab:{x,y,z}, grabArm:'L'|'R',  
  press:{x,y,z}, pressArm:'L'|'R' } ],  
  markers:[{name,x,y,z}], up:{x,z}, right:{x,z} }
```

T 恤固定 2 步（B 翻折 + C 对折）；裤子 1-3 步（左右对折 + tri 时两次长度折）。每步自带抓取臂与按压臂分工，行为树直接按 folds[] 顺序生成执行子树。

02 总体架构：ROS 2 节点拓扑与数据流

四层解耦：行为树只编排，不算几何、不做图像处理。任何一层可替换实现而不动其他层。

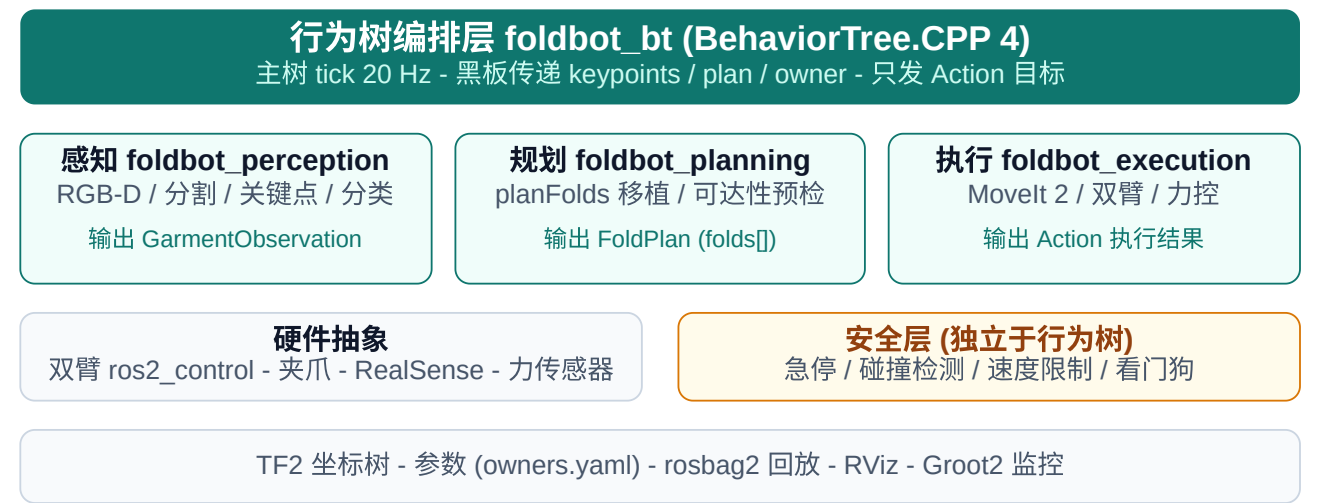


图 1 - 四层架构。行为树是唯一状态拥有者，感知/规划/执行为无状态服务。

为什么行为树不算几何

planner 是纯函数。行为树内嵌几何会让单测与调参被 tick 语义污染。规划做成 Service，行为树只问“给我计划”。

为什么用 Action 不用 Service

抓取/折叠是长耗时、可取消、需反馈的操作。feedback 承载“已接触/疑似滑移”，Reactive 节点靠它实时中止。

节点清单（第一阶段）

节点	职责	关键接口
foldbot_bt_main	加载主树 XML、tick、黑板、Groot2	第 03/04 节
perception_node	RGB-D 同步、分割、关键点、种类/尺寸	Action DetectGarment
planning_node	折叠计划、可达性预检、分类	Service PlanFold
execution_node	MoveIt 执行、双臂协调、夹爪、力控	Action ExecuteFoldStep
bin_manager_node	四箱层数、堆叠高度、码正偏移	Service GetPlacePose

03 行为树主树：任务循环、安全守卫与恢复

主树分三层：最外层是永不结束的任务循环，中间是安全守卫前缀，内层是单件衣服的完整处理子树。单件子树失败不会让整机停机，只会把该件衣服送入异常区并继续下一件。

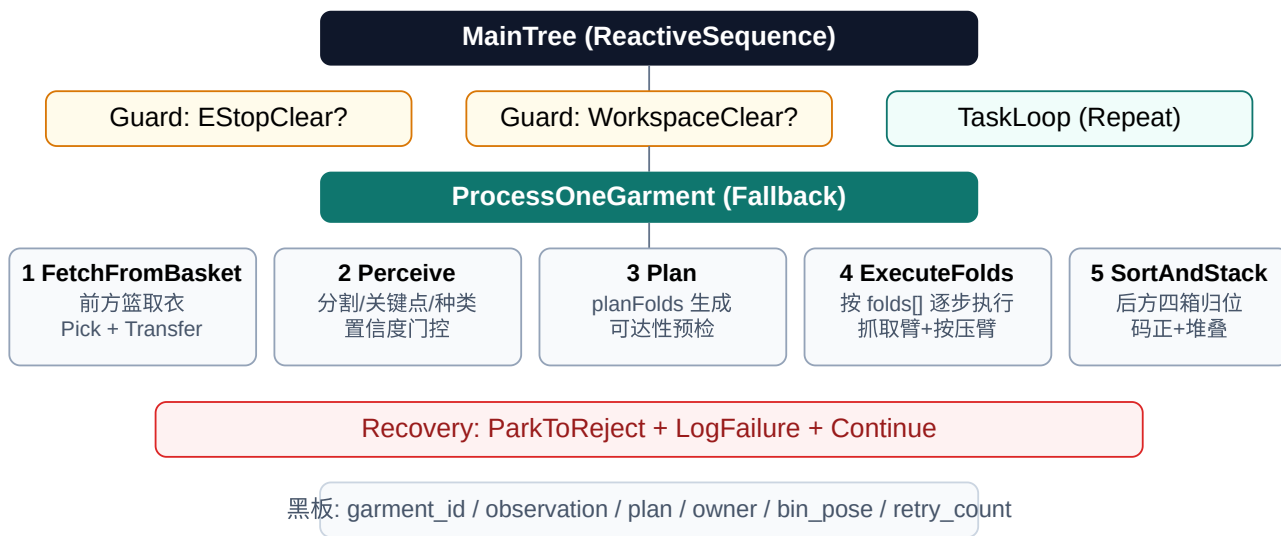


图 2 - 主树结构。Fallback 包住单件子树：任一步失败走 Recovery，不中断 TaskLoop。

设计要点

- **ReactiveSequence 做守卫**：急停或工作区有人闯入时，当前 Action 立即被 halt，机械臂停在当前位姿（由安全层接管减速停止），不是等当前步跑完。
- **单件子树可独立 tick**：调试时可单独加载 `ProcessOneGarment`，喂假的 observation 测规划与执行，不需要篮子有衣服。
- **重试计数在黑板上**：`retry_count` 按失败类型分开计（grasp / perception / plan），各自上限独立，避免抓取重试耗尽感知的重试额度。

04 行为树节点目录：每个叶子节点的契约

节点	类型	输入 (黑板)	输出 / 失败条件
IsBasketNotEmpty	Condition	-	篮内深度图有衣物团块返回 SUCCESS
PickFromBasket	Action (ROS)	basket_roi	输出 grasp_pose; 抓空 (力反馈无接触) 返回 FAILURE
PlaceOnTableCenter	Action (ROS)	table_center_pose	衣服落在中央台 ROI 内; 偏移过大返回 FAILURE 触发重新放置
DetectGarment	Action (ROS)	camera_frame	输出 observation (关键点+种类+尺寸); 置信度低于阈值返回 FAILURE
ClassifyOwner	SyncAction	observation.metric	调 planning 的 classify 逻辑, 输出 owner (dad/mom/daughter/son)
PlanFold	SyncAction	observation.keypoints	输出 plan (folds[]); 关键点缺失或不可达返回 FAILURE
ExecuteFoldStep	Action (ROS)	plan.folds[i]	逐步执行; 滑移检测触发返回 FAILURE 并请求重感知
VerifyFold	Action (ROS)	expected_outline	折后轮廓与预期 IoU 过低返回 FAILURE
GetPlacePose	SyncAction	owner	问 bin_manager 要码正后的放置位姿 (含当前堆高)
PlaceInBin	Action (ROS)	place_pose	后方衣箱放置; 碰撞风险返回 FAILURE 换偏移重试
ParkToReject	Action (ROS)	-	把失败衣物放异常区, 永不返回 FAILURE (兜底)

命名约定：Condition 以 Is/Has 开头，ROS Action 节点与第 05 节的 Action 名同名。行为树里出现一个新叶子节点，必须先在第 05 节注册接口，否则评审不通过。

黑板键清单（全局唯一，禁止节点私自加键）

```
garment_id: string      observation: GarmentObservation
plan: FoldPlan          owner: string (dad|mom|daughter|son)
bin_pose: PoseStamped  retry_grasp: int  retry_percept: int
last_error: string      fold_index: int (当前执行到第几折)
```

05 接口契约：msg / srv / action 先行冻结

接口冻结后，感知可用录制的 rosbag 假数据开发，规划可用现有 JS 单测的期望值做对照，执行可在 Gazebo 里先跑——三方并行，不互相等。

核心消息

```
# GarmentObservation.msg (对应 vision.js 的 findKeypoints 输出)
string kind                # shirt | pants
string length              # long | short | na (裤子用)
geometry_msgs/Point[] keypoints # 按 keypoint_names 索引
string[] keypoint_names    # sleeveL,sleeveR,shoulderL,...
float32 shoulder_width_m   # classifyOwner 的 metric (上衣)
float32 waist_width_m      # classifyOwner 的 metric (裤子)
float32 confidence         # 0..1, 低于阈值行为树拒绝规划
sensor_msgs/Image mask     # 分割掩码, 供 VerifyFold 做 IoU

# FoldPlan.msg (对应 planner.js 的 plan 输出, 字段一一对应)
FoldStep[] folds
geometry_msgs/Vector3 up
geometry_msgs/Vector3 right
string owner
```

```
# FoldStep.msg (对应 planner 的单个 fold step)
FoldParams params # px,pz,dx,dz,fold_sign,axis_height,duration
string label      # 翻折 | 对折 | 三折1 | 三折2
string grab_arm   # L | R
string press_arm  # L | R
geometry_msgs/Pose grab_pose
geometry_msgs/Pose press_pose
```

服务与 Action

接口	类型	说明
/planning/plan_fold	Service	GarmentObservation -> FoldPlan; 内部即 planFolds/planPantsFolds 移植
/planning/classify	Service	metric -> owner; 阈值读 owners.yaml, 第一阶段沿用现有 0.69/0.595/0.50 (肩宽)
/perception/detect_garment	Action	目标: 拍一张并检测; 反馈: 当前阶段 (segment/keypoint/classify); 结果: GarmentObservation
/execution/pick / place	Action	目标位姿 + 接近方向; 反馈: 距离/接触状态; 可取消
/execution/execute_fold_step	Action	输入单个 FoldStep; 双臂同时下发 (抓取臂轨迹 + 按压臂保持); 反馈含滑梯标志
/bin_manager/get_place_pose	Service	owner -> PoseStamped; 内部维护四箱 stack_height 与码正偏移 (复用现有堆叠逻辑)

冻结规则：接口字段只增不改。需要改字段语义时新建 v2 消息，旧字段保留一个版本周期。planner 的阈值 (肩宽/腰宽分界) 属于参数不是接口，可随时在 yaml 里标定。

06 感知子系统：从颜色分割到 RGB-D 学习型关键点

现在 (仿真) 的做法

合成图 + 已知主体色阈值分割 (segmentShirt); 凸包顶点 + 凹点聚类找袖口/肩/下摆 (findKeypoints); 裤子靠裆部凹点 (findKeypointsPants); 种类靠裆槽纵深判定 (detectKind)。在纯色、平铺、无褶皱时完美。

真机为什么会崩

花色衣服没有单一主体色; 褶皱让凸包顶点乱跑; 袖子压在身上时凹点消失; 光照变化让颜色阈值漂移。这套启发式在真机上只能当 fallback, 不能当主路径。

真机主路径 (三步)

1. **分割**: 学习型分割 (如基于 SAM 微调或 U-Net) 输出衣物 mask, 不再依赖单一颜色; RGB-D 深度通道辅助区分衣物与桌面 (高度差)。
2. **关键点**: HRNet 类关键点网络直接回归袖口/肩/下摆/腰/裆等关键点, 输出格式与 findKeypoints 完全一致 (同名 keypoint_names), 规划层无感切换。需要标注数据: 第一阶段可用仿真合成图 + 少量真机手标冷启动。
3. **尺寸测量**: 关键点像素坐标经手眼标定 (相机内参 + 桌面平面) 投影到桌面坐标系 (单位米), 再喂给 classifyOwner 的原有阈值逻辑。堆叠层数由深度图在衣箱 ROI 内的高度估计。

置信度与降级

- 关键点网络每个点输出置信度, 整体 confidence 取最弱关键点; 低于 0.6 行为树不进入规划, 转而请求换角度重拍或人工介入。
- 学习模型不可用 (权重缺失/推理超时) 时自动降级到 vision.js 移植的几何路径, 并在日志里打 `perception_mode=geometric`, 便于统计降级频率。
- VerifyFold 复用分割 mask: 折后实际轮廓与规划预期轮廓 (用 foldPointVert 推进 markers 得到) 做 IoU, 这是现有几何代码在真机上的第二个复用点。

手眼标定是硬前置: 没有标定, 关键点只是像素, planner 的 0.69 m 肩宽阈值无从谈起。M3 里程碑的标定误差要求: 桌面平面内小于 5 mm。

07 折叠规划子系统：planner.js 的 1:1 移植

这一层是整个迁移里最省事的：它本来就是纯函数。移植工作 = 把 JS 翻成 C++ (或先 Python 快速验证)，把单测一起翻过来，阈值抽到参数文件。

现有函数	真机签名	不变的东西
<code>planFolds(k)</code>	PlanFold (shirt)	ABC 三点定义、2 步 folds、grab/press 分工
<code>planPantsFolds(k, mode)</code>	PlanFold (pants)	左右对折 + tri/bi/none 的步数逻辑
<code>classifyOwner(kind, k)</code>	Classify service	肩宽阈值 0.69/0.595/0.50；腰宽 0.52/0.44/0.36 (米)
<code>detectPantsLength(k)</code>	感知后处理	长宽比大于 1.4 判长裤
<code>pantsFoldMode(owner, length)</code>	参数查表	成人长裤 tri、其余 bi、儿童短裤 none

新增的真机职责 (现有代码没有的)

- **可达性预检**：每个 FoldStep 的 grab_pose / press_pose 先过一遍解析 IK (kinematics.js 的 fk/ik 思路推广到真机臂的简化模型) 或 MoveIt 的 IK 服务；任一不可达，PlanFold 直接返回失败，行为树走重感知而不是执行到一半才发现够不着。
- **抓取点避让皱褶**：规划输出的抓取点若落在感知标记的皱褶区 (mask 局部法向突变)，沿折痕方向平移至最近平坦点。这是真机加的唯一一条规划后处理，不改动 folds 结构。

验收方式：现有 planner.test.mjs 的每个用例翻译成真机单元测试，输入同一组关键点，期望的 folds 数量、foldSign、grabArm 必须逐项相等。这一条不过，说明移植错了，不是真机调参问题。

08 运动执行子系统：MoveIt 2 + 力控收尾

这是 sim-to-real gap 最大的一层。仿真里 IK 算出关节角、衣服顶点用 `foldPointVert` 精确跟随；真机里衣服是 deformable object，抓哪、滑多少、折痕实际落在哪，全都不确定。执行层的设计目标不是复现仿真轨迹，而是每一步都可验证、可中止、可重来。

分层执行策略

阶段	做法	说明
接近	MoveIt 2 规划无碰撞路径到抓取点上方 5 cm	双臂互为碰撞体；按压臂先就位再抓取，避免时序竞态
接触	笛卡尔直线下探 + 力阈值停 (如 3 N)	不用位置硬压桌面；力反馈判定已接触布面
抓取	夹爪闭合 + 抬升 2 cm 试探	试探时监测力/夹爪开度：空夹 (开度接近 0) 判抓空
折叠搬运	沿折痕的弧线轨迹 (参考 <code>foldPointVert</code> 的旋转路径)	轨迹形状复用现有折叠几何，但速度降至准静态 (真机第一阶段宁慢勿快)
放下与按压	按压臂沿 <code>press_pose</code> 轻压 1-2 s	固定折痕、防回弹；力控制恒力而非恒位
验证	松开双臂、重拍一张给 <code>VerifyFold</code>	不验证不进入下一步；这是和仿真最大的流程差别

- **双臂协调：**一个 MoveIt planning group 管双臂，或两组 + 时序屏障；第一阶段用屏障 (按压臂到位才允许抓取臂动)，不做双臂协同轨迹优化。
- **视觉伺服 (第二阶段引入)：**搬运中以 10 Hz 跟踪被抓布角的实际位置，偏离预期折叠路径过大时减速或停下重抓。第一阶段先开环 + 步后验证。
- **解析 IK 的新角色：**只做预检与仿真对照，不进实时控制环。

09 工作空间、坐标系与 TF 树

用户已确认的工作流直接决定真机布局：待洗篮在机械臂前方（对面），折叠台在中央，四位家人的衣箱在后方（近操作者侧）。行为树里的 Fetch 与 Sort 分别对应这两个方向，位姿全部由 TF 给出，不在代码里写死数值。

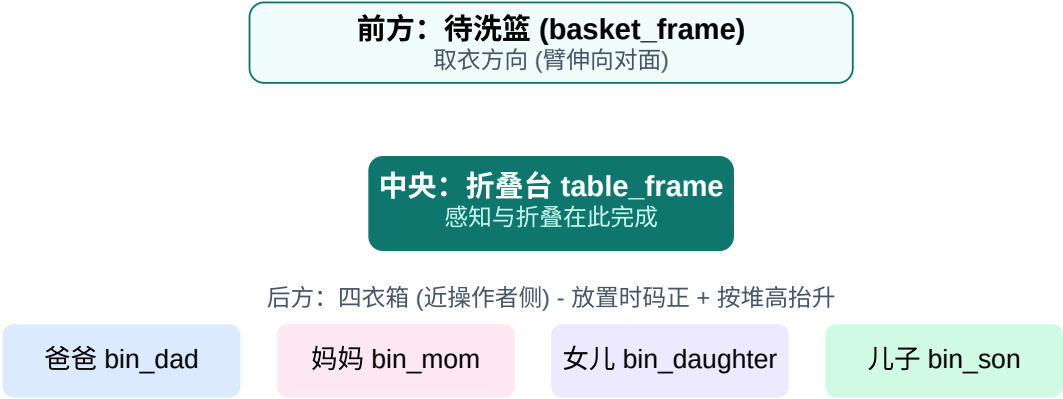


图 3 - 工作空间布局 (俯视)。方向与用户在仿真里确认的流程一比一对应。

TF 树 (静态部分)

```
map
|- base_link (双臂基座中点)
|   |- arm_left_base / arm_right_base
|   |- camera_frame (顶部 RGB-D, 手眼标定外参)
|- table_frame (桌面, z=0 平面)
|- basket_frame
|- bin_dad / bin_mom / bin_daughter / bin_son
```

- **码正堆叠**：bin_manager 维护每个箱的已放件数与当前堆高；放置位姿 = 箱中心 + 码正朝向 (与箱边平行) + 堆高偏移。这一逻辑直接对应仿真里“摆正方向、居中、逐层上叠”的现有实现。
- **标定流程**：相机对桌面 ArUco 板一次标定得到 camera->table 外参；篮与箱的位姿用示教 (手动引导臂触碰三点) 录入，存为静态 TF 发布参数。

10 异常恢复与安全设计

失败分类与恢复策略 (行为树 Fallback 统一收口)

失败类型	检测方式	重试策略	降级终局
抓空	夹爪开度/力反馈无物	抓取点沿轮廓内移 1 cm，最多 3 次	该件放异常区，记日志
关键点缺失/低置信	confidence 低于 0.6	换高度重拍 1 次 + 几何 fallback 1 次	请求人工铺平后重试
规划不可达	PlanFold 预检失败	把衣服在台面上旋转 15 度后重感知	放异常区
折叠滑移	VerifyFold IoU 过低	从当前步重感知、重规划剩余步，最多 2 次	展开重折 1 次，再失败放异常区
放置碰撞风险	MoveIt 规划失败	放置点在箱内网格换位 (最多 4 个候选)	提醒人工整理衣箱

安全设计 (双层)

- **行为树层 (软)**: 主树 ReactiveSequence 前缀的 Guard 节点持续检查急停状态与工作区侵入 (深度相机在安全区内检出人体)，任一触发立即 halt 正在执行的 Action。
- **硬件层 (硬)**: 急停按钮、安全光幕/碰撞检测走 ros2_control 的安全链，不依赖行为树 tick；速度上限在驱动层钳制 (协作模式)。行为树被杀掉，臂也必须能停。
- **上电自检**: 主树启动前强制跑一遍自检序列 (TF 完整、相机有帧、双臂回零、急停未按下)，不过不进入 TaskLoop。

原则: 任何重试都有次数上限，任何上限耗尽都有确定终局 (异常区或人工请求)。真机调试最怕的是无限重试把衣服越弄越乱——行为树的 Fallback 就是为此而设。

11 三阶段路线：仿真到真机的收敛路径

阶段一：只分拣不折叠

从前方篮取已叠好的衣服，识别种类/尺寸，放入后方对应箱并码正。刚性程度最高：衣服已是规整块，抓取与放置都是标准 pick-and-place。先验证 TF 标定、分类阈值、双臂可达域与整条行为树主干。

验收 连续 20 件分类正确率 100%，放置无碰撞

阶段二：辅助折叠

人把衣服在中央台铺平，臂执行完整 folds[] (T 恤 2 步、裤子 1-3 步)。引入步后 VerifyFold 与滑移重规划。这一阶段暴露 deformable 操作的真实失败率，决定要不要上视觉伺服。

验收 T 恤单件成功率不低于 80% (含重试)

阶段三：全自主

从乱衣堆抓取单件、抖开/铺平、再折叠分拣。需要抓取点策略与铺平原语 (抓角、拖平)，是独立的研究性子课题，不在本设计展开，只预留行为树扩展点 (FlattenGarment 子树)。

验收 单件全流程成功率不低于 60% 起步迭代

仿真先行 (每阶段都先在仿真过)

- **行为树与接口层**：直接在 ROS 2 里用假感知节点 (发布录制的 observation) 跑主树，验证 Fallback 与重试逻辑，不需要物理仿真。
- **运动层**：Gazebo / Isaac Sim 里用刚性方块代替折好的衣服测 pick-and-place；布料仿真 (MuJoCo / Isaac 布料) 只用于折叠轨迹的粗验证，不追求与真机一致。
- **感知层**：仿真渲染图做关键点网络的预训练 (domain randomization：颜色/花纹/光照/褶皱扰动)，真机少量手标微调。

为什么这个顺序：阶段一的每件衣服都是规整块，sim-to-real 几乎无 gap，能最快把行为树、接口、TF、安全这套骨架在真机上立起来。骨架立住后，阶段二只需换感知与执行的实现，行为树结构不动。

12 里程碑与验收标准

里程碑	交付物	验收标准 (可测)
M0	接口包 foldbot_interfaces + 行为树空主干 + 假节点	主树能 tick; 假感知喂数据能走完 ProcessOneGarment 全流程 (执行节点为仿真 stub)
M1	planning 移植完成 + 单测对照	现有 planner/fold/vision 单测的翻译版全绿; 同输入下 folds 逐项与 JS 版相等
M2	阶段一真机: 分拣闭环	连续 20 件已叠衣服: 分类正确 20/20, 放置成功不低于 19/20, 无安全触发误报
M3	感知主路径 (RGB-D + 关键点网络 v1)	手标测试集关键点误差中位数小于 1.5 cm; classifyOwner 用真机测量值分类正确率不低于 95%
M4	阶段二真机: 辅助折叠	T 恤含重试成功率不低于 80%; 裤子 (bi 模式) 不低于 70%; VerifyFold 能拦住故意折错的样本
M5	连续运行与异常收口	混合 30 件连续运行不死锁; 每类失败都在重试上限内收口到异常区或完成, 有完整 rosbag 日志

开发顺序建议

1. 先 M0 把骨架立起来 (接口 + BT + 假数据), 所有人对着同一棵行为树说话。
2. M1 与 M3 可并行: 规划移植不依赖硬件, 感知先用仿真图预训练。
3. 硬件到位后先 M2 立真机闭环, 再投入折叠 (M4); 不要在分拣没跑通时调折叠。

A 附录 A：行为树 XML 骨架（可直接加载）

BehaviorTree.CPP 4.x 语法。自定义节点在 foldbot_bt 里注册，ROS Action 节点通过 RosActionNode 基类桥接第 05 节接口。

```
<root BTCPP_format="4">
  <BehaviorTree ID="MainTree">
    <ReactiveSequence>
      <IsEStopClear/>
      <IsWorkspaceClear/>
      <Repeat num_cycles="-1">
        <Fallback>
          <SubTree ID="ProcessOneGarment"/>
          <Sequence>
            <Action ID="ParkToReject"/>
            <Action ID="LogFailure"/>
          </Sequence>
        </Fallback>
      </Repeat>
    </ReactiveSequence>
  </BehaviorTree>

  <BehaviorTree ID="ProcessOneGarment">
    <Sequence>
      <RetryUntilSuccessful num_attempts="3">
        <Sequence>
          <Action ID="PickFromBasket"/>
          <Action ID="PlaceOnTableCenter"/>
        </Sequence>
      </RetryUntilSuccessful>
      <RetryUntilSuccessful num_attempts="2">
        <Action ID="DetectGarment"/>
      </RetryUntilSuccessful>
      <Action ID="ClassifyOwner"/>
      <Action ID="PlanFold"/>
      <SubTree ID="ExecuteAllFolds"/>
      <Sequence>
        <Action ID="GetPlacePose"/>
        <Action ID="PlaceInBin"/>
      </Sequence>
    </Sequence>
  </BehaviorTree>

  <BehaviorTree ID="ExecuteAllFolds">
    <Sequence>
      <!-- 按 plan.folds 动态展开：每步 = 执行 + 验证 -->
      <RetryUntilSuccessful num_attempts="2">
        <Sequence>
          <Action ID="ExecuteFoldStep"/>
          <Action ID="VerifyFold"/>
        </Sequence>
      </RetryUntilSuccessful>
    </Sequence>
  </BehaviorTree>
</root>
```

注：ExecuteAllFolds 在真机实现里由代码按 folds[] 长度动态生成同构子树（或用 BT 的循环装饰器遍历黑板里的 plan），XML 里先以单步形态占位冻结结构。

B 附录 B：ROS 2 包结构与文件迁移对照

```
foldbot_ws/src/  
  foldbot_interfaces/      # msg/srv/action, 第 05 节冻结  
  foldbot_bt/              # 主树 XML + 自定义节点 (C++)  
  foldbot_planning/        # fold.js + planner.js 的 C++ 移植  
  foldbot_perception/      # RGB-D + 关键点网络 + 几何 fallback  
  foldbot_execution/       # MoveIt 2 配置 + 执行 Action 服务器  
  foldbot_bringup/         # launch + TF + 参数 (owners.yaml)  
  foldbot_sim/             # Gazebo/Isaac 场景 + 假感知节点
```

现有文件	迁移到	动作
fold.js	foldbot_planning/src/fold_geometry.cpp	翻译 + 单测对照
planner.js	foldbot_planning/src/planner.cpp	翻译 + 阈值进 owners.yaml
kinematics.js	foldbot_planning/src/reachability.cpp	思路保留，做可达性预检
vision.js	foldbot_perception (几何 fallback)	翻译；主路径另建学习模型
garments.js 常量	foldbot_bringup/config/owners.yaml	参数化；纹理仅 foldbot_sim 用
app.js 状态机	foldbot_bt 主树 XML	状态机翻行为树 (第 03 节)

开工检查清单

- ROS 2 (Humble/Jazzy) + MoveIt 2 + BehaviorTree.CPP 4 + Groot2 环境就绪
- 双臂 + RGB-D 的 ros2_control / 驱动能单独 jog 与拍照
- 手眼标定完成，桌面平面误差小于 5 mm
- 现有 JS 单测的期望值导出为对照数据 (JSON)，供 M1 验收

下一步：按 M0 先搭 foldbot_interfaces 与 BT 空主干，用假感知数据把主树跑通——这一步不需要任何硬件，是整份设计里风险最低、最该先做的一件。